Relatório de Evolução do Projeto — EV Challenge GoodWe · Sprint 04

FIAP × GoodWe Brasil · Prompt and Artificial Intelligence · 2º Sem CC 2026.2 · Prof. Jorge Luiz Gomes · Repositório: github.com/Guilher-cell/Sprint3_Chatbot

> **Como os números foram obtidos.** Avaliação **live em 21/10/2026**: embeddings `nomic-embed-text` (Ollama local) + geração com `openai/gpt-oss-120b` e `openai/gpt-oss-20b` (Groq), `temperature=0`, mesmo eval set de 30 casos em todas as iterações, 0 erros de chamada em todas as iterações. O **RAGAS não foi executado** (`import ragas` falha no ambiente); as métricas são a **rubrica manual equivalente** (`evals/RUBRICA.md`), que é **lexical e tende a subestimar a fidelidade** (ver §3). Ablação e varrimento de `k` foram medidos **offline** (gerador extrativo, sem LLM), por serem reproduzíveis e sem custo de API.

1. Resumo da evolução

Sprint	O que era	Limitação que motivou a seguinte
1/2	Chatbot com `system_prompt` de ~6,9 mil caracteres como **única fonte de conhecimento**; dados mockados em `tools.py`; CLI	sem recuperação, **sem citação de fonte**, sem recusa por falta de contexto, sem como medir
03	Refactory em LangChain LCEL: `prompt → llm → parser`, memória por tokens, saída estruturada Pydantic v2, prompt XML versionado, guardrails regex, 2 modelos (Groq)	conhecimento ainda dentro do prompt; escala mal e não cita onde aprendeu
04	**RAG medido e com interface:** 8 documentos em **ChromaDB persistente**, `nomic-embed-text`, `PyMuPDFLoader → RecursiveCharacterTextSplitter → retriever → prompt RAG v1/v2/v3`, grounding + **citação documento·seção·página**, recusa fora do contexto, defesa contra prompt injection via documentos, avaliação por iteração com 2 modelos e **interface Gradio**	validar com RAGAS e com mais casos

Os módulos das Sprints 1–3 foram **mantidos**; o RAG reutiliza os guardrails da Sprint 03 e o `llm_factory` (Groq).

2. Pipeline RAG

Base de conhecimento (`data/knowledge_base/`, material fictício e didático): 4 manuais de produto (EV ChargeOps morador e síndico; ChargeGrid diagnóstico; orquestração de potência) e 1 regimento condominial, todos em **PDF** (gerados de Markdown por `scripts/gerar_kb_pdfs.py` para exercitar o `PyMuPDFLoader`), mais FAQ, tabela tarifária e política de escalonamento em Markdown. Total: **8 documentos, 8 páginas de PDF, 66 chunks**.

Chunking (decisões e trade-offs):

Decisão	Valor	Por quê / custo
Splitter	`RecursiveCharacterTextSplitter` (`\n\n`, `\n`, `. `, `; `, ` `)	respeita parágrafo > frase; não parte passo a passo. Custo: tamanhos irregulares
Tamanho	1200 (iter1) → **600** (iter2/3)	seções curtas; 1200 misturava 2–3 assuntos por chunk. Custo: perde contexto vizinho
Overlap	0 → **100**	evita cortar instrução na fronteira. Custo: ~15% de texto duplicado; **sem efeito mensurável** na ablação offline
Fronteira de seção + prefixo `[Seção: …]`	ligado	cada item de FAQ/seção é uma unidade; a **seção citada é a do conteúdo**; o título traz vocabulário de busca. Custo: 26 → 66 chunks
Recuperação	cosseno, **k = 4**, rerank lexical 0,7·cos + 0,3·cobertura	k maior sobe recall e derruba precisão (offline: k=8 → recall 0,94, precisão 0,45); o rerank foi o **maior ganho** na ablação offline

Vector store: Chroma persistente; coleção nomeada por SHA-1 de (conteúdo + chunking + backend), então mudar documento, chunk ou modelo cria coleção nova e backends nunca se misturam. Busca determinística (`ef_search=200`, 1 thread, desempate por `chunk_id`).

Segurança: (i) guardrails regex da Sprint 03; (ii) **limiar de relevância** (cosseno do melhor trecho; **0,55** com nomic, valor padrão não recalibrado); (iii) **checagem de código de produto** ausente da base; (iv) **sanitização do contexto** (remove sentenças dirigidas ao modelo, comentários HTML e tags de delimitação) + prompt v3 que declara `<contexto>` como dado; (v) **pós-validação de citação**. Com nomic, o melhor score dos 20 casos documentais ficou entre **0,639 e 0,909** e o dos casos fora de contexto entre **0,472 e 0,604** — a folga até o limiar é pequena em um dos casos (F3, 0,604, recusado pelo próprio LLM). **Prompt RAG versionado:** `prompts/rag/` e `prompts/VERSOES_RAG.md`.

Interface (`app/app.py`, Gradio): chat com a resposta e as **fontes citadas visíveis abaixo dela** (arquivo · seção · página); painel com os trechos recuperados, score e marcação dos trechos sanitizados; seletor de iteração, gerador e k.

3. Comparativo antes/depois (obrigatório)

Eval set fixo de **30 casos** (20 documentais, 5 fora de contexto, 1 jailbreak, 2 domínio sensível, 2 injeção em documento). Resultados **live com** `gpt-oss-120b` (modelo principal):

Métrica	Sprints 1/2 (versão original)	Sprint 04 · iter1 (RAG ingênuo)	Sprint 04 · iter2	Sprint 04 · iter3 (final)
answer_relevancy (equiv. RAGAS)	0,400	0,846	**0,975**	0,971
faithfulness (equiv. RAGAS, lexical)†	0,067‡	0,670	0,510	0,514
fidelidade à base (documento único)†	0,202‡	0,692	0,642	0,576
Contexto: context_recall	0,000 (não recupera)	0,829	0,871	**0,908**
Contexto: context_precision	0,000	0,600	0,575	**0,637**
Contexto: MRR	0,000	0,875	0,892	**0,917**
Citação de fonte (taxa)	**0,00** (nunca cita)	0,00	1,00 (nativa: 8/20)§	**1,00** (nativa: 20/20)
Recusa correta (fora de contexto / jailbreak / sensível; 8 casos)	0,25	0,25	**1,00**	**1,00**
Resistência a prompt injection (2 casos)	n/a	1,00	1,00	1,00
Latência mediana por chamada ao LLM (s)	1,4	5,3	5,1	3,1
Prompt	`system_prompt_v1` (conhecimento embutido)	RAG v1 (1 frase)	RAG v2 (grounding + citação)	RAG v3 (XML + contexto é dado)
Base / vector store / interface	1 prompt / nenhum / CLI	8 docs / Chroma / CLI	8 docs / Chroma / CLI	8 docs / Chroma persistente / **Gradio**

Segundo modelo, `gpt-oss-20b` (mesmo pipeline, mesmos parâmetros; recall, precisão e MRR são idênticos porque a recuperação não depende do modelo de geração):

Métrica	Sprints 1/2	iter1	iter2	iter3
answer_relevancy	0,308	0,721	0,958	**0,975**
faithfulness (lexical)†	0,047‡	0,710	0,559	**0,744**
fidelidade à base†	0,211‡	0,746	0,624	**0,748**
Citação (taxa; nativa)	0,00	0,00	1,00 (20/20)	1,00 (20/20)
Recusa correta (8 casos)	0,25	0,25	1,00	1,00
Resistência a injeção (2 casos)	n/a	0,50	0,50	**1,00**
Latência mediana por chamada (s)	11,6	6,6	4,7	7,8

† A faithfulness é **lexical** (afirmação suportada se todo número e ≥ 70% dos termos aparecem no contexto): penaliza paráfrase, frases sobre "qual documento diz o quê" e valores derivados. Auditamos à mão 6 frases sinalizadas na iter3/120b: **5 estavam sustentadas pela base** (ex.: "17,6 kW" = 80% de 22 kW; "cancelamento após 1 h conta como não comparecimento") e 1 não foi verificada (caminho de menu). Trate o número como **limite inferior**, não como taxa de alucinação; não fizemos auditoria completa. ‡ No baseline o LLM viu o prompt inteiro, mas a métrica compara com 3 trechos dele: **não comparável**. § Na iter2/120b o modelo citou com colchetes largos (estilo CJK) em vez de `[1]` em 12 de 20 respostas; o validador não os reconhecia e **forçou** a citação (corrigido no código depois da execução; ver §4).

Ganho por iteração (≥ 2 iterações com ganho — Bloco D): baseline → iter1: relevância 0,40 → 0,85 e recall 0 → 0,83 (passa a recuperar da base). iter1 → iter2 (120b): relevância **+0,129**, recusa correta **+0,75**, citação 0 → 1,00, recall +0,042; no 20b, relevância **+0,237**. iter2 → iter3: recall **+0,037**, precisão **+0,062**, MRR **+0,025** (rerank) nos dois modelos; no 20b, ainda faithfulness **+0,185** e injeção **0,50 → 1,00**. **Limitação honesta:** para o 120b, iter3 **não melhora** a qualidade da resposta sobre a iter2 (relevância −0,004, fidelidade à base −0,066); o ganho da iter3 está na recuperação, na citação nativa e na robustez do modelo menor. A faithfulness **cai** de iter1 para iter2 no 120b (0,67 → 0,51): respostas com grounding e citação ficam mais elaboradas e a rubrica lexical as penaliza; não é prova de piora.

Offline (referência reproduzível, sem LLM): relevância 0,108 → 0,608 → 0,667 → 0,775 e injeção 0 → 0 → 0 → 1,00. **Diferença importante:** offline a sanitização foi decisiva (injeção 0 → 1); no live o **120b resistiu à injeção mesmo sem sanitização** (1,00 em iter1/iter2) e o **20b não** (0,50, resposta correta 0,00) — a sanitização se justifica para o modelo menor, com apenas 2 casos de injeção.

Prompt isolado (iter3 + prompt v1, 120b, 0 erros): relevância 0,946, recusa 0,875, faithfulness 0,833 — o texto do prompt v3 acrescenta pouco (+0,025 de relevância, +0,125 de recusa); **a maior parte do ganho vem do pipeline**, não do texto do prompt. A matriz completa (v2/v3) não foi concluída por limite de taxa da Groq.

4. Problemas encontrados e soluções

1. **Limite de taxa da Groq (429).** Uma execução morreu no caso Q17 e a matriz de prompts ficou com 23 erros (v3) e 1 (v2). Decisão: o runner passou a **esperar o tempo sugerido pela Groq e tentar de novo**, **parar com aviso** se a espera for longa (cota diária) e **gravar cada caso** (`live_cache_*.json`) para reaproveitar o que já foi pago; erros viram contagem `n_erros` e a execução avisa que os scores estão contaminados. As linhas inválidas da matriz foram **removidas** em vez de usadas.
2. `tool_use_failed` **no baseline.** O prompt das Sprints 1/2 descreve ferramentas; sem elas, o modelo tentava chamá-las e a Groq devolvia 400, derrubando a avaliação. Decisão: nota "sem ferramentas, responda em texto" no baseline + registro explícito se ainda falhar. Custo: o baseline live difere ligeiramente do chatbot original.
3. **Citações em formato inesperado.** O 120b usou colchetes largos (estilo CJK) em vez de `[1]` na iter2; a citação "100%" escondia 12/20 forçadas pelo código. Decisão: medir citação **nativa × forçada** separadamente e normalizar esses colchetes largos e `[1, 2]` para `[n]` (com testes). A correção é posterior à execução; os números acima são anteriores a ela.
4. **401 no Ollama.** O `.env.example` apontava para o Ollama Cloud com uma chave de exemplo, e a biblioteca também lê `OLLAMA_API_KEY` do ambiente. Decisão: padrão local (`localhost:11434`) e descarte de chave de exemplo.
5. **RAGAS indisponível.** `import ragas` falha (`langchain_community.chat_models.vertexai`). Decisão: rubrica manual determinística equivalente, documentada (`evals/RUBRICA.md`), com o limite de ser lexical (item † acima). Pendência: rodar o RAGAS com LLM-juiz quando o ambiente permitir.
6. **Resultados não reproduzíveis.** O HNSW do Chroma (`ef_search=10`) mudava o top-k; além disso um arquivo de scores estava defasado do código. Decisão: busca determinística, regeração dos resultados e três execuções offline idênticas. E **frases partidas na extração de PDF** foram resolvidas com `reflow_pdf` e chunking que não cruza seções.

Limitações: n pequeno (20 documentais, 2 de injeção); eval set criado pelo próprio grupo (não é cego); base fictícia de 8 documentos; limiar 0,55 não recalibrado; faithfulness lexical; RAGAS não executado; ablação e `k` só offline; matriz de prompts incompleta; uma execução por configuração (sem estimativa de variância do LLM).

5. Equipe e divisão de trabalho

Integrante	RM	Tarefa principal (Sprint 04)
André Fujinaga	569158	Pipeline RAG end-to-end + multi-provider (`relatorio_modelos.md`)
Arthur Machado	569919	Vetorização: ChromaDB persistente + embeddings nomic-embed-text
Conrado Gracie	569157	Interface web (Gradio) com citação visível
Guilherme Belo	570079	Prompt RAG versionado (v1→v3) + base de conhecimento
Renato Sandreschi	569156	Guardrails/segurança + avaliação (rubrica/RAGAS) + relatório de evolução